SYSTEM ANALYSIS DOCUMENTATION (SAD):

BIOFIN ORACLE

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

BioFin Oracle *is the strategic solution designed to address the "Triple Volatility" crisis (biological, meteorological, and market volatility) currently impacting Malaysian SME orchard owners in the high-value Musang King durian sector. This document transitions farm management from subjective, experience-based methods to a data-driven, executive decision-making framework.*

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the ***BioFin Oracle*** *development. It details the integration of the* ***ILMU-GLM-5.1*** *reasoning engine with agricultural telemetry to provide deterministic action commands.*
The key elements of the system that have been covered are as follows:

1. ***Architecture****:*
    a) *It is a cloud-native, stateless platform. The system is structured as a* ***Next.js full-stack monolith*** *with a React frontend dashboard and Next.js API Routes serving as the orchestration layer. Long-running inference tasks are streamed to the client via* ***Server-Sent Events (SSE)****, resolving serverless timeout constraints without a separate async worker process. The reasoning engine operates as an external service layer through the* ***ILMU-GLM-5.1*** *API*
2. ***Data Flows:***
    a) *Data from orchard sensors (NPK, soil moisture) and user-uploaded CSV/JSON records flows through* ***REST API*** *endpoints. The data is summarized into trend signals, processed by the* ***ILMU-GLM-5.1*** *reasoning engine, and finally streamed back to the user via SSE, returning deterministic action commands and simulation updates to the user dashboard. All session data is strictly ephemeral; no persistent database is used.*
3. ***Model Process:***
    a) *The model shows the end-to-end workflow of core agentic features: ingesting multi-modal data, performing specialized reasoning through Sensory, Risk, and Market agents, and executing critical logistics commands such as "Advance Harvest".*
4. ***Role of Reference:***
    a) *This document is a reference for all developers and testers to enable the overall high-level architecture of this project, specifically the integration of the* ***ILMU-GLM-5.1*** *service layer and the emergency fallback logic.*

1.2. Background:

Malaysian SME orchard owners in the high-value agricultural sector, specifically Musang King durian producers, currently face a "Triple Volatility" crisis involving biological, meteorological, and market factors. Historically, these operations have been managed through subjective, "experience-based"

© UMHackathon 2026

farming, which leads to 15–20% annual budget waste due to preventive over-fertilization and up to 40% yield collapse from missed meteorological windows.

1. *Previous Version:*
    a. *Traditional orchard management is operated through manual, intuition-led processes where data is fragmented and decision-making is subjective. In this "experience-based" model, farmers lack real-time visibility into soil NPK metrics or international market supply surges, often resulting in catastrophic capital loss during extreme weather events or competitor price dumping.*
2. *Changes in Major Architectural Components*
    a. **Transition to Cloud-Native Orchestration:** Moved from manual tracking to a **Next.js full-stack monolith** architecture utilizing **dual-pipeline data ingestion** for both structured sensor data and unstructured trade news.
    b. **LLM Service Layer Integration:** Replaced subjective human intuition with the **ILMU-GLM-5.1** reasoning engine, which enforces deterministic JSON schemas for all agricultural action commands.
    c. **SSE Streaming Pipeline:** Implemented **Server-Sent Events (SSE)** within the Next.js API route to stream multi-step reasoning progress (parsing, summarising, searching, analyzing) in real-time to the client, preventing serverless timeout failures during complex weather and market simulations — without requiring a separate async worker process.

3. *New capabilities that are only introduced in this*
    a. **Agentic Reasoning Ledger:** Specialized evaluation nodes—Sensory, Risk, and Market Agents—that provide explicit causal explainability for every command, such as "Advance Harvest by 48 Hours".
    b. **Digital Twin Simulation Sandbox:** Interactive modules allowing users to manipulate variables for bio-cultivation optimization, Weather-at-Risk (YaR) modeling, and supply chain arbitrage.
    c. **Automated Market Intelligence:** Real-time web-scale ingestion of competitor supply data and global trade reports via the **Tavily Search API**.
    d. **SME Compliance Automation:** Integrated rule-scanning for **LHDN MyInvois 2026** digital signatures and ROI estimation based on automated labor efficiency gains.

1.3. Target Stakeholder

Stakeholders	*Roles*	*Expectations*

© UMHackathon 2026

Farm CEO	*High-level decision maker*	*Wants explicit commands (e.g., "Advance Harvest") based on sensor fusion.*
CFO	*Financial risk manager*	*Needs cash flow runway forecasting and market supply shock hedging.*
Operations Manager	*Field implementation lead*	*Need interactive simulations to test labor and fertilizer trade-offs.*
Development Team	*System maintainers*	*Clear API usage contracts and modular, well-documented architecture.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Full-stack Web Dashboard (Next.js/React)*
Architecture	*Client-Server model with a Next.js App Router orchestration layer connecting frontend simulations to a specialized backend intelligence pipeline*

BioFin Oracle *is structured as a client-server-based system where the primary client is the* ***Next.js Web Dashboard****. The client communicates via a* ***Backend API layer (Next.js API routes)*** *which acts as the central orchestrator. The backend manages dual-pipeline data ingestion, coordinating between the* ***ILMU-GLM-5.1 Reasoning Engine****, the* ***Tavily Search API*** *for market intelligence. All session data is strictly ephemeral — the system operates without a persistent database.*

2.1.2 LLM as Service Layer

The **ILMU-GLM-5.1** is integrated as a core reasoning engine rather than a generic module. It transforms multi-modal agricultural telemetry into deterministic JSON structures that drive the system's simulation state and critical harvest commands.

2.1.3 Dependency Diagram

e. ***Prompt Construction:*** *Prompts are constructed by a backend orchestration layer that aggregates summarized sensor telemetry (NPK, soil pH, moisture) and unstructured search results into a strictly formatted JSON system prompt. What goes into the context window?*

f. ***Context Window:*** *The context window contains compact* ***Trend Labels*** *(e.g., fertilizerTrend), recent operations signals (e.g., specialEventCount, irrigationVolume), and real-time snippets from the* ***Tavily Search API*** *regarding competitor supply.*

g. ***Receiving & Parsing:*** *The system receives raw LLM output, passes it through a* ***sanitiseResult*** *repair function to ensure schema integrity, and hydrates the* ***Next.js*** *frontend state via React hooks.*

h. ***Token Enforcement:*** *Token limitations are enforced in the pre-aggregation stage; raw CSV records are chunked and summarized into statistical trends (last 3 records vs. overall mean) before reaching the* ***ILMU-GLM-5.1****.*

i. ***Major API Calls:*** ** Frontend Dashboard calls POST /api/analyze.*

© UMHackathon 2026

- Backend calls `api.ilmu.ai/v1/chat/completions` for reasoning.
- Backend calls `api.tavily.com/search` for live market intelligence.

j. ***System Interactions:*** *The **Next.js** frontend manages interactive simulations; the backend interacts with external AI/Search microservices for logic, streaming all progress and results back to the client via **SSE**. The system is entirely stateless with no persistent data store.*

k. **Distributed Components:** The architecture is planned for deployment across **Vercel/AWS EC2** for the application server. No managed database service is required; the stateless, serverless architecture stores all data ephemerally within the request lifecycle.

2.2.2. Sequence Diagram (User Journey: Weather Risk Mitigation)

a. ***Walkthrough:*** *The user journey begins with the SME Farmer (CEO) uploading historical weather logs to assess imminent risks to the Musang King yield.*

b. ***Step-by-Step Interactions:***

1. *User -> Next.js UI: Farmer uploads operationsData.csv (containing weather event records) via the dashboard.*
2. *Next.js UI -> API Route: Frontend sends a POST request with file data to /api/analyze.*
3. *API Route -> Logic Engine: Backend executes summariseOperations() to detect special farming events and fetchMarketIntelligence() to query the Tavily API for localized weather insurance data.*
4. *API Route -> ILMU-GLM-5.1: Logic engine sends a structured prompt containing sensor telemetry and search snippets to the AI reasoning engine.*
5. *ILMU-GLM-5.1 -> API Route: AI returns a deterministic JSON object classifying the risk as "rain" and providing a causal reasoning ledger.*
6. *API Route -> Next.js UI: Backend sanitizes the JSON and hydrates the dashboard state.*

c. *Start-to-Finish Demonstration: The dashboard displays a "Critical Action" alert: "Advance Harvest by 48 Hours," allowing the farmer to execute logistics before the storm surge causes root rot or fruit-drop.*

2.2. Technological Stack

2.2.1. ***Frontend: Next.js (React)*** *with **Sora** and **JetBrains Mono** typography. It utilizes **Lucide-React** for iconography and **Chart.js** for probability density and yield simulations.*

2.2.2. ***Backend: Next.js API Routes*** *(Node.js) acting as the sole orchestration layer for all AI services and data processing. No additional backend framework is used; the monolith is self-contained.*

© UMHackathon 2026

2.2.3. ***Data Storage: None (Ephemeral)*** *— all data processing is strictly in-memory within the serverless function's request lifecycle. No persistent database, cache layer, or task queue is used in the current implementation. (PostgreSQL and Redis are identified as Future Roadmap components for multi-session trend analysis.)*

2.2.4. ***Cloud/Deployment:*** *Application served via* ***Vercel*** *or* ***AWS EC2****. Static assets and agricultural data file backups can optionally leverage* ***AWS S3*** *for agricultural data backups.*

2.3. Key Data Flows

2.3.1. ***Data Movement:*** *Raw data is ingested as multi-modal streams (CSV, JSON, and Web Search), processed into statistical trend vectors, and transformed into high-impact "Trade-off Decisions" by the reasoning engine.*

2.3.1.1. Data Flow Diagram (DFD)

a. ***Movement:*** *Farmer uploads data -> readFile() parses contents -> summariseEnvGeo/BioCrop/Operations/Financial() generates JSON summaries -> callZAI() generates decision logic -> sanitiseResult() repairs schema -> Dashboard displays output.*

b. ***Proper Laws:*** *The system avoids "Black Holes" by ensuring every data ingestion (e.g., financialData) results in a specific UI update (e.g., salesInsights card).*

2.3.2. **Normalized Database Schema**

a. **Logical Model (3NF):** The schema is designed to prevent data redundancy across multi-orchard operations. For example, the Records table acts as a junction to map specific telemetry types (NPK, Soil Moisture, pH) to a unique Orchard_ID and Timestamp, ensuring 3NF integrity.

3. Functional Requirements & Scope

This section focuses on the high-impact ***BioFin Oracle*** *MVP, which prioritizes the "Agentic Decision Ledger" and the "Digital Twin Simulation Sandbox" to demonstrate technical feasibility to the judging panel.*

3.1. Minimum Viable Product:

3.1.1. ***Core Features for UMHackathon 2026 Demo***

#	Features	Description
1	**Multi-Modal Ingestion Portal**	Supports CSV/JSON uploads for environmental & geospatial data (GPS, soil NPK/pH), biological & crop data, farming operations data (inputs, irrigation, events), and financial & commercial data.
2	**ILMU-GLM-5.1 Decision**	Processes ingested data to generate deterministic action commands (e.g., "Advance

© UMHackathon 2026

	Engine	Harvest") and causal explainability via Sensory, Risk, and Market agents.
3	**Digital Twin Simulation Sandbox**	Interactive sandbox allowing farmers to manipulate sliders (fertilizer, labor, market allocation) to see real-time impact on profit and bio-health.
4	**SME Compliance & ROI Auditor**	Automated scans for LHDN MyInvois 2026 e-invoicing compliance and ROI payback estimation based on labor efficiency gains.
5	**Transparency & Evidence Feed**	Live telemetry monitoring and real-time market news aggregation using the Tavily Search API.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirements	Implementation
Scalability	BioFin Oracle must handle large multi-year agricultural datasets and concurrent simulation requests.	Use **SSE streaming** to progressively emit stage results and avoid timeout failures, and **Next.js** orchestration to scale reasoning tasks efficiently.
Reliability	The system must provide actionable baseline insights even if the ILMU-GLM-5.1 API is unreachable.	Integrated `X-AI-Fallback` header and `buildDefaultResult` logic to return a deterministic "Golden Release" baseline.
Maintainability	AI reasoning, market intelligence, and dashboard states must remain modular and independently updatable.	Separate microservice-style logic in **Next.js API routes** with strict TypeScript interfaces for all data structures.
Token Latency	ILMU-GLM-5.1 API calls must return a response within 5s at p95 under normal load.	Use **SSE streaming** for progressive stage delivery and input summarization (**Trend Labels**) to reduce token count and processing time.

© UMHackathon 2026

Cost Efficiency	Average token costs per session must stay within budget by optimizing prompt length.	Inputs are **summarized into trends** (e.g., last 3 records vs mean) and repeating market queries are **cached for 24 hours**.

3.3. Out of Scope / Future Enhancements

The team has identified the following high-level features for the "Grand Vision" of BioFin Oracle that are not part of the current UMHackathon 2026 scope:

a. ***Direct IoT Hardware Integration:*** *Real-time, automated data streams from physical soil NPK and moisture sensors directly into the dashboard (currently using CSV/JSON uploads).*
b. ***Autonomous Logistics Execution:*** *Automated API trading or the autonomous execution of logistics contracts and export shipping orders based on AI recommendations.*
c. ***Global Benchmarking Pipeline:*** *A comprehensive multi-year historical training pipeline for multi-orchard comparative benchmarking and predictive yield modeling across regional borders.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: New AI decision modules (e.g., the "Risk Agent" or specialized "Pest Outbreak" nodes) will be released to 5% of beta-testing Farm CEOs in the initial post-hackathon pilot. After monitoring for decision accuracy and system stability, the rollout will gradually scale to 100%. A/B testing will be conducted to determine which LLM prompting strategies (e.g., Chain-of-Thought vs. Agentic) perform better for yield-at-risk predictions.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: In terms of recovery for the BioFin Oracle system, if ILMU-GLM-5.1 API anomalies are detected (e.g., malformed JSON or timeouts), ER is triggered. The automated pipeline deploys the* ***Golden Version*** *of the application, utilizing the deterministic buildDefaultResult baseline to ensure the farmer still has access to safe operational defaults.*

4.1.3. ***Priority Matrix:***

4.1.3.1. ***P1 Critical:*** *If Soil Moisture > 85% AND Storm Probability > 90% simultaneously, the system triggers an emergency harvest command to preserve capital.*

4.1.3.2. ***P2 High:*** *If bioHealthIndex drops below 55, the system prioritizes yield-at-risk calculations and triggers insurance gap evaluations.*

4.2. Monitoring:

4.2.1. ***System Health Monitoring:*** *The system monitors the health of external dependencies (ILMU-GLM-5.1 and Tavily Search API). Monitoring triggers a fallback if token latency exceeds a 5s threshold or if sensor telemetry (e.g., NPK levels) enters a "Critical" zone, sending immediate alerts to the user dashboard.*

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

a. ***Internet Connectivity:*** *Users have stable internet connections on their devices during the multi-modal reasoning phase of the ILMU-GLM-5.1 API.*

b. ***Device Availability:*** *Farm owners have access to digital devices (tablets or laptops) to interact with the Next.js simulation sandbox and "Digital Twin" modules.*

c. ***Data Representation:*** *The synthetic CSV/JSON datasets provided as proxies accurately represent regional Musang King orchard conditions and Malaysian market volatility.*

4.4. External Dependencies: *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	Purpose	Risks
ILMU-GLM-5.1 API	Core reasoning engine for multi-step financial inference and generating deterministic action commands.	**High** - Downtime disables causal logic. Mitigation: Implement `X-AI-Fallback` to return a "Golden Release" baseline.
Tavily Search API	Real-time web search aggregator for market prices, competitor supply, and international trade news.	**Medium** - API failure limits the "Market Agent's" ability to detect immediate supply surges.
Meteorological Webhooks	Provides real-time storm trajectories and precipitation data for "P1 Critical" triggers.	**Medium** - Inaccurate data could trigger unnecessary emergency harvest logistics.

© UMHackathon 2026

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days, following a compressed two-week sprint methodology.

5.1. *Project Timeline:*

- **Day 1–2 (Planning):** Finalize **SAD** and **TypeScript interface contracts** (biofin.ts) to establish the shared data foundation and SSE event shapes.
- **Day 3–4 (Backend):** Develop **Next.js API Routes** with SSE streaming and **CSV/JSON** parser logic for telemetry ingestion.
- **Day 5–6 (AI Integration):** Engineer **ILMU-GLM-5.1** prompts and implement JSON schema validation for deterministic output.
- **Day 7–8 (Frontend):** Build the **Next.js** dashboard and **Simulation Sandbox** with real-time state hydration.
- **Day 9 (Testing):** Validate the **Emergency Rollback** logic and fallback header response for rural resilience.
- **Day 10 (Pitch):** Finalize the "Grand Vision" presentation for the **UM Hackathon 2026** judging panel.

5.2. *Team Members Role:*

Tan Liang Chuan: *Lead AI Orchestrator – Responsible for ILMU-GLM-5.1 logic, prompt engineering, and the agentic reasoning ledger.*

Thet Htun Oakar & Ng Yi Ren: *Backend Lead – Focused on Next.js API Routes, SSE streaming pipeline, and CSV/JSON parser logic for telemetry ingestion.*

Tan Li Hong & Chen Bing Yan: *Frontend Lead – Responsible for the Next.js interactive simulation dashboard and Chart.js yield projections.*

5.3. *Recommendations:*

- **Scalability:** Introduce **persistent session storage (e.g., Redis or PostgreSQL)** as a future roadmap item to enable multi-session trend analysis and reduce repeated API overhead for frequently accessed market news.
- **Reliability:** Extend the existing **SSE streaming + buildDefaultResult fallback** pattern with proactive health checks against the ILMU-GLM-5.1 endpoint to trigger graceful degradation before timeout windows are reached, maintaining p95 latency targets under load.
- **Performance:** Investigate **Edge Computing** or local SQLite caches for rural orchards with intermittent 4G/5G connectivity.

© UMHackathon 2026